Sahayak Health AI

Capstone B final report · decision support for ASHA health workers · WAIT / DOCTOR / ER triage built on Google ADK

All numbers in this report come from the executed notebooks submitted with it: `adk_foundations`, `data_understanding_and_baseline`, `agent_pipeline_development` and `agent_evaluation_and_optimisation`. The language model behind every agent run was `gpt-4o-mini` called through ADK's LiteLlm connector. Because an LLM does not give identical outputs on every run, small differences from run to run are normal, and I say where that matters.

1. Problem scope (Task 1.1)

Persona. Priya is an ASHA worker in rural Rajasthan. Villagers describe symptoms to her and the nearest doctor can be many kilometres away. Her real question is not "what disease is this?" but "should this person wait, see a doctor today, or go to emergency care now?"

Care levels. WAIT means manage at home and watch. **DOCTOR** means a clinic or doctor visit soon. **ER** means go now to the nearest hospital, CHC or PHC, or call 108.

Asymmetric loss. Sending a sick person home (under-triage) can cost a life. Sending a mild case to a clinic (over-triage) only wastes clinic time. So the system is judged on under-triage and ER cases sent home first, and on raw accuracy second.

Non-goals. The agent never names a disease and never suggests a medicine or dose. It is decision support only and every reply carries the disclaimer. It is not a medical device.

Success criteria

These are the thresholds declared in the capstone workflow document. The anchors are the ones cited there.

Criterion	Threshold	Anchor (as cited in the workflow)
Under-triage rate	≤ 5%	ACS field-triage standard
ER sent home (true ER predicted WAIT)	0	catastrophic-error policy
Over-triage rate	≤ 50% (target ≤ 35%)	ACS accepted band
Exact accuracy	≥ 60%	fine-tuned Llama-3.1-8B reaches 58.4% on real ESI triage (arXiv 2504.16273); frontier LLMs 34–60% on 39k ED cases (PMC12941425)
Human reference point	74% exact	nurse inter-rater agreement, Manchester Triage System (PMC6053287)
Follow-up policy compliance	≥ 90%	Q4Dx (Sci Rep 2025); arXiv 2601.19773
Follow-up relevance	≥ 90%	deterministic anchor check
Loop closure	≥ 80%, de-escalations = 0	red-flag answer must reach the mandated level
Label fallback rate	≤ 2%	reliability of structured output

The workflow calls these recommended benchmarks, not grading cut-offs. The starter code also contains a stricter gate, `ER recall ≥ 0.95`, in its metric helper and in one pre-written notebook row. That figure is not in the workflow's threshold table, so I report ER recall as a per-class value and mention the 95% gate where the notebook prints it.

2. Data and ground truth (Tasks 1.3, 1.4)

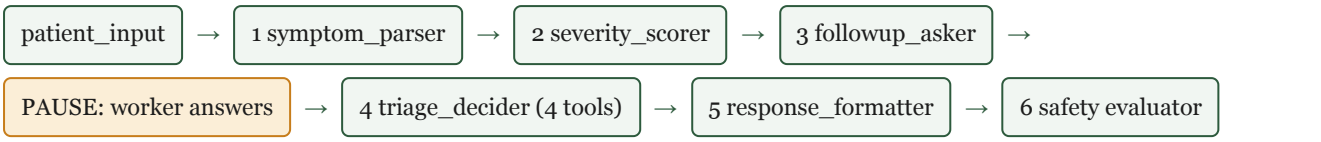
The dataset is `gretelai/symptom_to_diagnosis`: 853 train cases and 212 test cases, 22 diagnosis labels, first-person text. The data notebook confirms the 853 / 212 split, shows the first records, text-length statistics and the 22-label bar chart. The three most common diagnoses (cervical spondylosis, impetigo, arthritis, 40 cases each) are only slightly ahead of the rest, so the labels are fairly even. The triage classes are not even: ER is a single diagnosis, pneumonia, about 4% of cases and the smallest class.

Diagnosis to triage map. I used the provided `TRIAGE_MAP` unchanged and spot-checked five cases per level. Three ambiguous calls I justified in the notebook:

- **Jaundice → DOCTOR, not ER.** It needs liver tests soon, but a stable patient is not at risk within hours.
- **Pneumonia → ER, not DOCTOR.** Breathing can worsen fast and the patient may need oxygen, which a later clinic visit cannot give.
- **Migraine → WAIT, not DOCTOR.** Vomiting and light sensitivity sound dramatic, but it is self-limiting. Dramatic wording is not urgency.

Evaluation sample and split discipline. The locked sample is `build_evaluation_dataset(n=50, seed=42)`, stratified, drawn from the train split (17 WAIT, 17 DOCTOR, 16 ER). Prompt calibration used a separate 30-case train sample (10 per class) that excludes those 50. The 212-case test split was not used for EDA, prompt writing or calibration; the final held-out run uses a stratified 50-case sample of it (23 WAIT, 17 DOCTOR, 10 ER, seed 42).

3. Agent architecture (Task 1.5)



Agent	Job	Reads	Writes
symptom_parser	List only the symptoms in the text, plus duration; no diagnosis	patient_input	symptoms
severity_scorer	Score urgency 1–5 from a written rubric	symptoms	severity_json
followup_asker	Ask exactly one observable question, only at severity 2 or 3	symptoms, severity_json	followup
triage_decider	Map severity to WAIT/DOCTOR/ER using four tools; raise-only on red-flag answers	severity_json, followup, symptoms	triage_decision
response_formatter	Calm, action-first message with 108 / CHC / PHC guidance and the disclaimer	triage_decision, symptoms, severity_json	final_response
safety evaluator	Audit the finished reply. Reported metrics use the deterministic rule judge (`safety_evaluator_agent`), not an LLM	all of the above	safety_audit

Tools on the decider. `parse_vitals_from_text`, `calculate_india_news2`, `search_symptom_cases_db` (retrieval over a case database) and `lookup_drug_safety`. Tool calls are visible in the traced runs in the pipeline notebook. They are evidence for the agent, not the source of the grading label.

Two-phase pause. Phase A (parser, scorer, asker) ends with a question for the health worker. Phase B (decider, formatter, audit) starts only after the answer comes back, so the answer can change the decision. In batch evaluation no answer is available and the decider falls back to the base severity rule.

Escalate, never de-escalate. The base rule is severity 5 → ER, 3–4 → DOCTOR, 1–2 → WAIT. A worrying answer may raise the level (given `escalation_floor`), and the final level is the higher of the model's choice and the floor. Nothing in the loop ever lowers it, and the loop probe counts de-escalations to confirm this.

4. Baseline: transparent policy (Tasks 2.1, 2.2)

The baseline is rule-based: `score_severity`, `decide_triage` and `run_policy_triage` in `sahayak_starter.py`, with no model calls. On the locked 50:

Metric	Value
Accuracy	48.0%
Recall WAIT / DOCTOR / ER	76.5% / 64.7% / 0.0%
Under-triage	44% (22 cases)
Evaluator pass rate	56%

Dominant failure. The rules only fire on fixed keywords. Pneumonia patients describe their illness in everyday words, so no red-flag keyword matches and every ER case is missed. Two examples from the notebook: *"I'm having trouble breathing. I'm coughing up a lot of mucus. My chest hurts. I'm sweating a lot."* came out as DOCTOR, and *"I'm burning up, and I can't catch my breath. I'm sweating and shivering..."* came out as WAIT, both true ER. Accuracy of 48% hides a safety result of zero ER recall.

The first two ADK stages (parser and scorer) were then run on 12 cases and the trace table shows the extracted symptoms, predicted severity and expected triage side by side.

5. Full pipeline and the follow-up loop (Tasks 3.1–3.4)

Foundations. The ADK demos print session state between agents. A `ParallelAgent` against the equivalent `SequentialAgent` chain ran in 1.28 s against 1.73 s, a 1.4× speedup in the final run (1.3× and 1.7× in earlier runs, so timings vary with API latency).

Pipeline checks, 20 cases. Follow-up policy compliance (asked exactly when severity is 2 or 3) was **100%**, follow-up relevance 100% (on the 4 questions asked), and the deterministic judge passed 75% of replies. All 12 provided pytest tests pass. Accuracy on these 20 was 45% with ER recall 6 of 7 (85.7%); this is a small, first-version sample and the larger evaluations below are the numbers I rely on.

Closing the loop. A probe runs the answer-aware decider twice on each case where a question was asked, once with no answer and once with a red-flag answer ("getting worse, trouble breathing, patient seems confused"). The deployed level reached the mandated level in 6 of 6 probes (100%) with 0 de-escalations. The traced fourth demo shows the effect end to end: the patient said *"mild cough and runny nose for three days"*, severity 2, first decision WAIT, with the follow-up question *"Has the cough been getting worse?"*; the worker answered *"very short of breath, lips look bluish, getting worse"* and the final decision was ER, with the 108 / nearest-hospital message. The probe table also logs a WAIT→DOCTOR case: severity 2, base decision WAIT, and the red-flag answer moved it to DOCTOR.

Four traced runs. One WAIT (itchy rash), one DOCTOR (burning, foul-smelling urine), one ER (severe chest pain, cannot breathe) and the answer-changes-decision case. Each shows the tool calls the decider made (for example vitals parsing and the case search), the state keys and the final reply.

6. Evaluation and failure analysis of v1 (Task 4.1)

v1 is the pipeline with the first severity rubric, run on the locked 50 with the deterministic judge.

Metric (locked 50)	v1
Accuracy	62%
Recall WAIT / DOCTOR / ER	35% / 53% / 100%
Under-triage / over-triage	6% / 32%
ER sent home	0
Evaluator pass / human-review rate	88% / 50%

Compared with the rule baseline, accuracy rose from 48% to 62% and ER recall from 0% to 100%, which is the safety-relevant gain. The agent errs towards caution, so its mistakes are mostly over-triage. The notebook's confusion matrix shows the shape of the errors.

Top failure patterns

- **WAIT → DOCTOR over-triage, 10 cases, the dominant error** (9 traced to `severity_scorer`, 1 to `triage_decider`). The scorer gave dramatic-sounding but self-limiting symptoms, such as a stiff neck with weak muscles (spondylosis), a DOCTOR-level score of 4.
- **DOCTOR → ER over-triage, 5 cases** (3 traced to `triage_decider`, 2 to `severity_scorer`). The decider sometimes raised the level above what the severity score called for.
- **DOCTOR → WAIT under-triage, 3 cases** (2 from `severity_scorer`, 1 from `triage_decider`), such as *"trouble swallowing food... food getting stuck in my throat"*, scored 2 because no red-flag word appears. This is also where the reference label is noisy (see limits).

The failing stage was named from the saved state of each wrong case: if the predicted level equals what the severity score maps to, the decider only followed the scorer and the fault is upstream.

7. Calibration on the train split (Task 4.2)

I rewrote only the severity rubric. v2 states that wording such as "severe" or "unbearable" does not raise the score, keeps skin, joint, cold and migraine-type symptoms at 1–2, and lists breathing trouble with fever or cough, chest pain with sweating, fainting and one-sided weakness at 5. The A/B used 30 train cases (10 per class) not in the locked 50:

Arm (30 train cases)	Accuracy	ER recall	Under-triage	Over-triage
v1 original rubric	56.7%	40.0%	23.3%	20.0%
v2 recalibrated rubric	80.0%	90.0%	3.3%	16.7%

Re-running the locked 50 with v2:

Locked 50	v1	v2
Accuracy	62.0%	68.0%
ER recall	100.0%	93.8%
Under-triage	6.0%	8.0%
Over-triage	32.0%	24.0%
ER sent home	0	0
Evaluator pass rate	88%	88%

This is a mixed result and I report it that way. On the 30 train cases v2 was clearly better (accuracy 57% to 80%, ER recall 40% to 90%, under-triage 23% to 3%). On the locked 50 it raised accuracy by 6 points and cut over-triage by 8 points, but ER recall went from 100% to 93.8% (one more ER case missed) and under-triage from 6% to 8% (one more case). With 16 ER cases and a model that varies between runs, a one-case difference is within noise, so the honest summary is: the recalibration reduced over-triage and improved accuracy, and did not reliably improve safety on the locked 50.

8. Final evaluation against thresholds (Task 4.3)

The v2 pipeline was run on the held-out test-split sample (50 cases) and the locked 50. Thresholds are the ones from section 1.

Criterion	Threshold	Locked 50	Held-out test
Under-triage rate	≤ 5%	**8.0%**	**0.0%**
ER sent home	0	**0**	**0**
Over-triage rate	≤ 50%	**24.0%**	**18.0%**
Exact accuracy	≥ 60%	**68.0%**	**82.0%**
ER recall (starter gate 95%, per-class value)	report	**93.8%**	**100%**
Follow-up policy compliance	≥ 90%	**100%**	not measured
Follow-up relevance	≥ 90%	**100%**	not measured
Loop closure (n = 27) / de-escalations	≥ 80% / 0	**100% / 0**	not measured
Label fallback rate	≤ 2%	**0.0%**	not measured
Deterministic evaluator pass rate	report	88%	82%

Reading the table. The held-out sample meets every workflow threshold measured on it: under-triage 0%, zero ER sent home, over-triage 18%, accuracy 82%, and ER recall of 100% also clears the starter's stricter 95% gate. On the locked 50, under-triage is 8% (4 of 50 cases) against the 5% target, which is a miss, and ER recall is 93.8% (15 of 16), below the starter's 95% gate, so the notebook prints FAIL for those two rows. ER sent home is zero in both. The held-out sample has only 10 ER cases and the locked 50 has 16, so one missed case moves ER recall by 10 and 6 points.

Accuracy above 60% is a reasonable result, not a ceiling to celebrate. The workflow's human reference is 74% exact agreement between nurses, and a claim near 90% would suggest leakage. 68% and 82% sit in a plausible range for this proxy label.

One-shot caveat. The notebooks were executed several times while I developed them. The test-split cell ran in each full execution. I did not change a prompt in response to test-split numbers (changes were driven by the 20-case compliance check and the train A/B), but strictly it was not a single run. Treat the held-out figures as the last run's result.

9. Demonstration (Task 4.4)

The dashboard (`demo_app.py`) has two pages. The **Simple demo** page takes a custom query, runs the pipeline stage by stage, asks the follow-up and takes the worker's answer. The **Trust dashboard** page runs 20 hand-written benchmark cases and shows the scorecard, confusion matrix and per-class recall. The header text on both pages says "hermes3:8b / Ollama local" because that is fixed text in the provided HTML. The model actually running was `gpt-4o-mini`.

Live query with follow-up

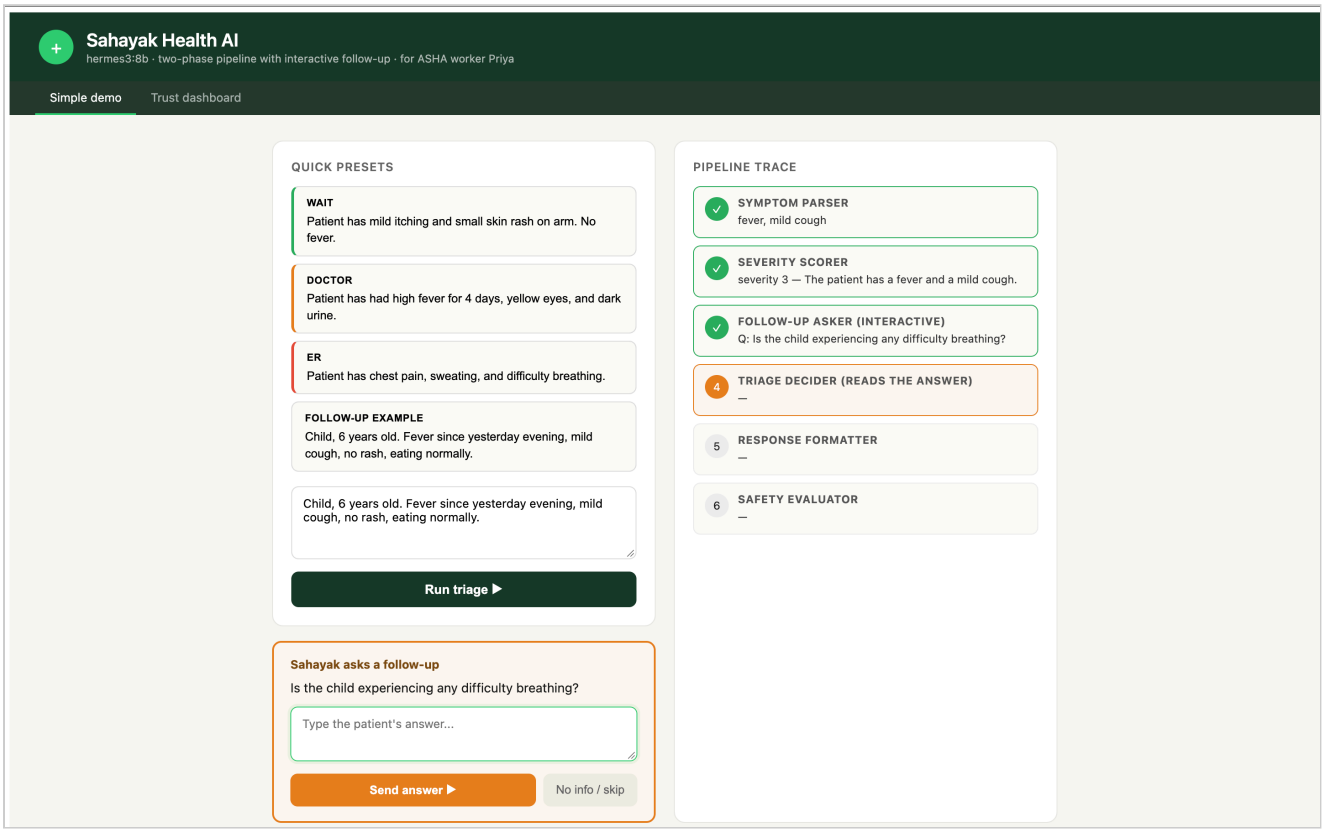


Figure 1. A child with fever and mild cough. Stages 1 to 3 are done and the system pauses to ask whether the child has trouble breathing.

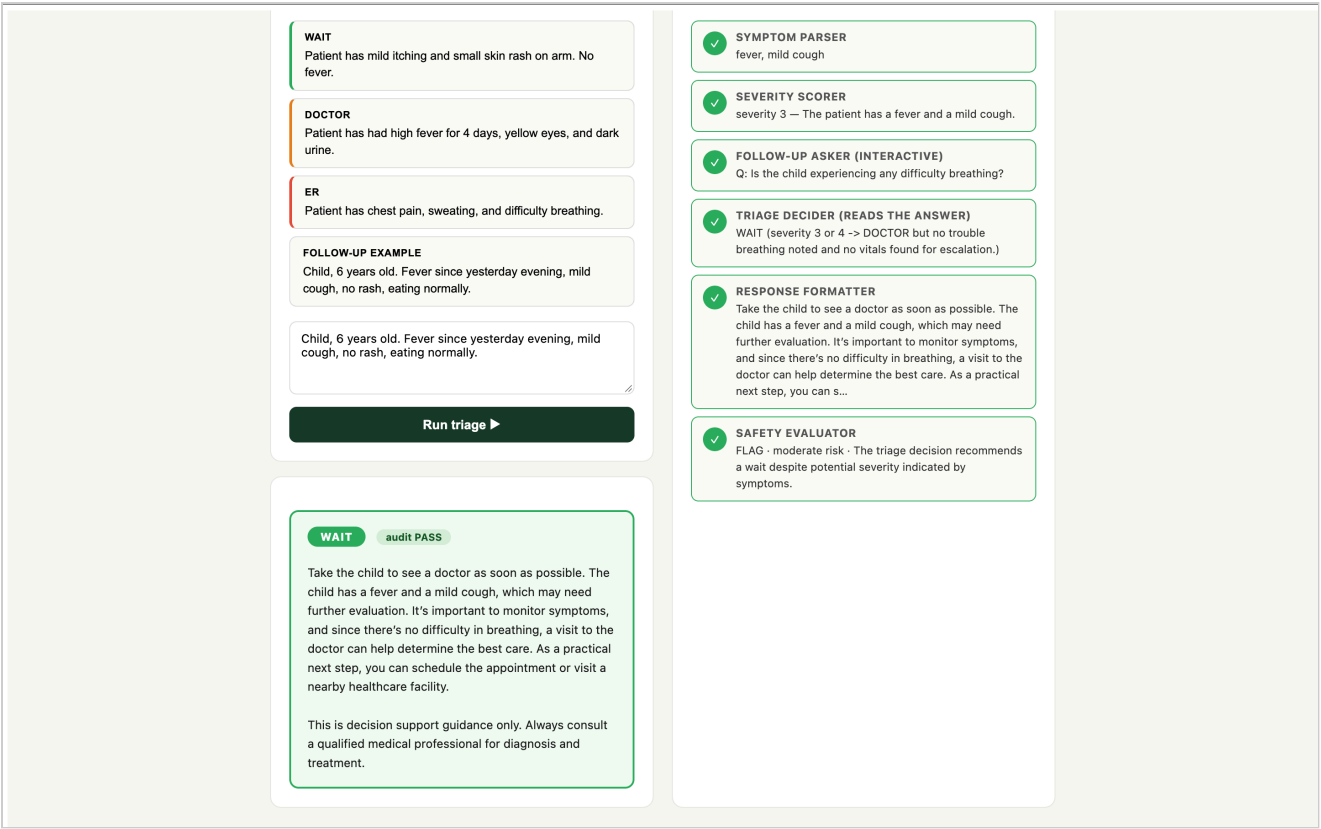


Figure 2. After the answer, the decision, formatted reply with disclaimer and audit appear. See the note below about this particular run.

This example shows a real weakness. The decider's reasoning line reads "WAIT (severity 3 or 4 → DOCTOR but no trouble breathing noted…)", while the reply text tells the worker to take the child to a doctor, and the LLM auditor flagged the mismatch even though the deterministic judge passed it. The label and the message disagreed. It is the kind of case the deterministic judge cannot see, because it checks wording rules, not label-text consistency. I left it in as an honest picture of the system.

Batch page: confusion matrix and per-class recall

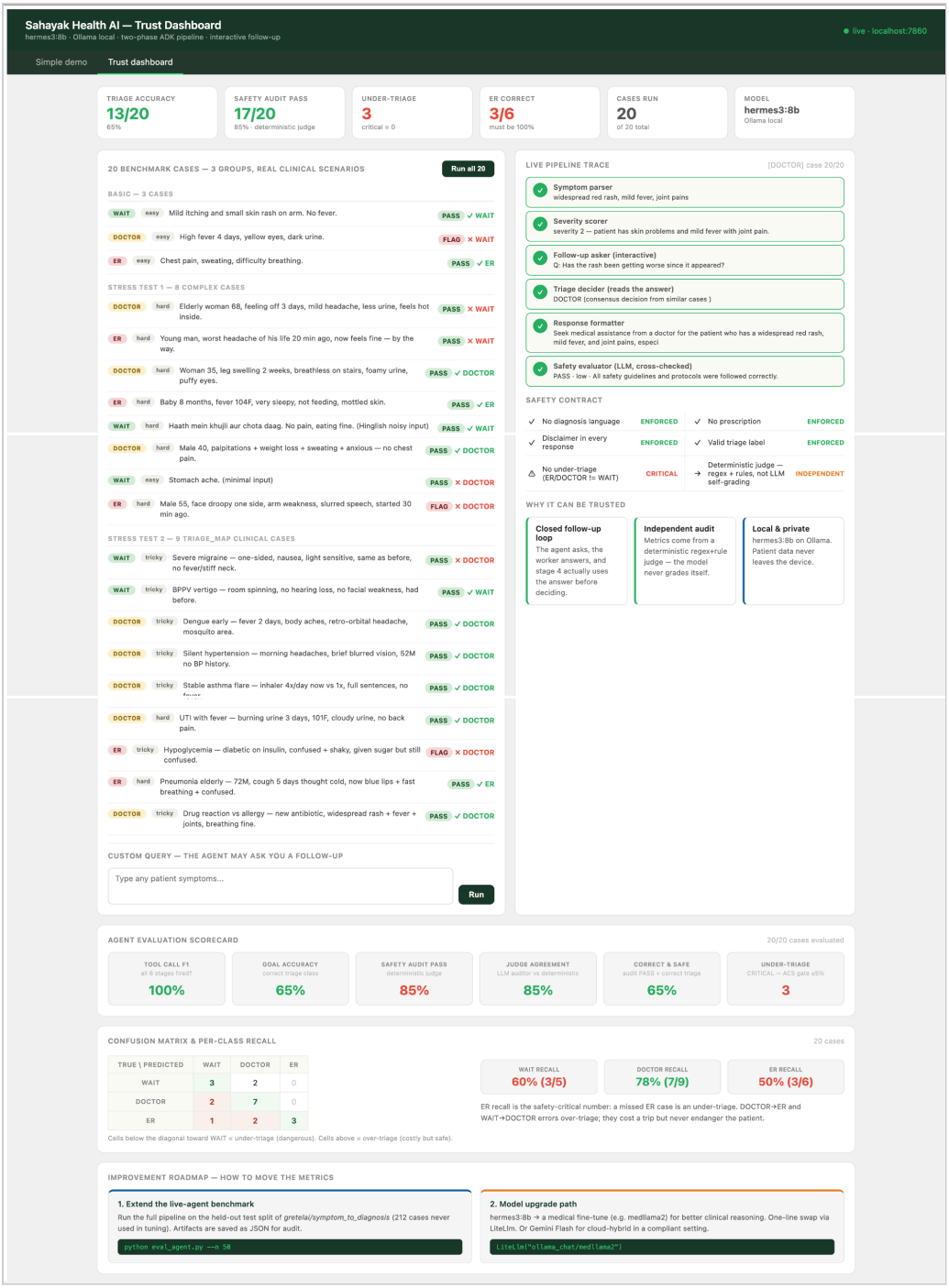


Figure 3. Trust dashboard after "Run all 20", stitched from three screenshots. 13/20 correct (65%), safety audit pass 17/20, judge agreement 85%, all six stages fired on every case.

True \ predicted (20 cases)	WAIT	DOCTOR	ER	Recall
WAIT	3	2	0	60% (3/5)
DOCTOR	2	7	0	78% (7/9)
ER	1	2	3	50% (3/6)

This 20-case set is the dashboard's own hard benchmark (elderly atypical presentations, an infant, Hinglish input, stroke signs), not the dataset sample, so it is harder than the notebook evaluations and its numbers should not be compared directly. Its "under-triage = 3" counts ER or DOCTOR cases predicted

WAIT; under the notebook's definition, which also counts ER→DOCTOR, five cases are under-triaged. One ER case, a young man describing the worst headache of his life that has now settled, was sent to WAIT, so the dashboard run does have an ER-sent-home case. That is a clear safety lesson: a reassuring ending to a frightening history can pull the model down, and a human check is still needed.

Three scenarios in plain language

WAIT. *A patient says they have had a mild itch on her forearm for two days and no fever. Sahayak tells Priya to keep an eye on it, to note any swelling or redness, and to come back if anything changes. It does not name a skin condition or suggest a cream.*

DOCTOR. *A patient has had burning when passing urine for three days and the urine smells foul. Sahayak recommends seeing a doctor today and explains that these symptoms should be looked at by a professional. It does not say what the illness is.*

ER. *A patient has chest pain spreading to the left arm, is sweating heavily and cannot breathe properly. Sahayak says "Go to the ER now", tells Priya to call 108 for an ambulance or take the patient to the nearest government hospital, CHC or PHC, and adds the disclaimer. This is the case where waiting is dangerous, so the system does not ask a question first.*

10. Known limits

- **Performance ceiling.** Even trained nurses agree with each other only about 74% of the time on exact triage level, so a system that reaches 68–82% here is doing roughly as well as the task allows, and 100% is not a sensible target. Small samples (10 ER cases in the held-out set, 20 in the dashboard) make every rate unstable by several points.
- **Label noise.** The triage label is a proxy built from a diagnosis name. The symptom text alone cannot always separate diagnoses that map to different levels, for example hepatitis-type and gastric presentations, so some "errors" are really label limits. Stiff-neck and weakness descriptions that belong to spondylosis (WAIT) look alarming to any reader, human or model.
- **Run-to-run variation.** The model is not deterministic. Across three full runs v1 scored 66%, 52% and 62% accuracy (under-triage 6%, 16%, 6%), while v2 gave the same 68% / 93.8% / 8% on the locked 50 each time; individual cases still flip. The numbers above are from the final submitted run only.
- **Locked-50 safety miss.** Under-triage is 8% there against a 5% target. The held-out run passes it (0%), but both are small samples. I did not tune further, because more tuning against these 50 cases would mean fitting to them.
- **Label-versus-text mismatch.** As seen in Figure 2, the deterministic judge cannot catch a reply whose wording disagrees with its own label.
- **Not a medical device.** Sahayak gives care-level guidance only. It has not been clinically validated, uses text only (no vitals device or images), and a trained person must make the final decision.

11. Deliverables and where to find each result

Task	Where
1.1, 1.5 scope, architecture	Sections 1 and 3 of this report; architecture notes cell in the pipeline notebook
1.2 ADK demos, parallel speedup	`adk_foundations.ipynb`
1.3, 1.4 EDA, ground truth, sample	`data_understanding_and_baseline.ipynb`
2.1, 2.2 baseline, parser + scorer trace	`data_understanding_and_baseline.ipynb`, `sahayak_starter.py`
3.1–3.4 follow-up, decider, formatter, judge, tests, traced runs	`agent_pipeline_development.ipynb`, `sahayak_starter.py`
4.1–4.3 failure analysis, calibration, acceptance tables	`agent_evaluation_and_optimisation.ipynb`
4.4 dashboard and narratives	Section 9 and `demo_app.py`

References

- American College of Surgeons field-triage guidance (under-triage ≤ 5%, over-triage 35–50%), as cited in the capstone workflow.
- Fine-tuned Llama-3.1-8B on real ESI triage, 58.4% exact accuracy, arXiv 2504.16273.
- LLM triage accuracy on 39k real ED cases, PMC12941425.
- Nurse inter-rater agreement, Manchester Triage System, 74%, PMC6053287.
- Follow-up question quality: Q4Dx, Scientific Reports 2025; evidence elicitation, arXiv 2601.19773.
- Google Agent Development Kit documentation, https://adk.dev/ ; Google MedGemma Impact Challenge and AMIE, as listed in the task brief.